

Explainable Occupation Classification on a DBpedia Person Subgraph: Diagnosing and Fixing Label Leakage in a Relational GCN

[Team Member 1 Name] ([imt account], [matriculation number]) · [Team Member 2 Name] ([imt account], [matriculation number]) · [Team Member 3 Name] ([imt account], [matriculation number])

Mini Project, Explainable Artificial Intelligence — Dr. Stefan Heindorf — Paderborn University

Abstract

We build an explainable node-classification pipeline on a DBpedia subgraph of 1,600 persons drawn evenly from four occupation classes (Athlete, Politician, Artist, Scientist), predicting occupation from a person’s local knowledge-graph neighbourhood with a Relational Graph Convolutional Network (R-GCN). An initial version of the pipeline reached a suspicious 100% train and test accuracy; we traced this to label leakage caused by including the exact `rdf:type` triple that defines each person’s class as a graph edge. After removing that triple, accuracy remained at 99.8%, still implausibly high. A controlled, five-condition ablation study isolated the true cause: DBpedia’s broader type hierarchy (redundant YAGO/Wikidata subtypes) still encoded the label even after the exact class triple was removed, while several other suspected shortcut predicates were tested and ruled out. Removing `rdf:type` entirely dropped accuracy to a defensible 62.9%; we then legitimately recovered performance to 73.8% by adding ten genuinely informative, non-label-defining predicates and tuning the R-GCN’s depth, width, and regularization via a controlled hyperparameter sweep. We explain the resulting model’s predictions with three complementary explainers — GNNExplainer, PGExplainer, and a custom Monte-Carlo Shapley-value approximation of SubGraphX — which converge on the same human-readable justifications for individual predictions, and we evaluate all three quantitatively with fidelity, sparsity, and stability metrics.

Keywords: Explainable AI · R-GCN · GNNExplainer · PGExplainer · SubGraphX · DBpedia · Label Leakage

1 Introduction

Graph Neural Networks (GNNs) are the standard architecture for learning on relational data such as knowledge graphs, but they remain black boxes: predictions cannot be read off the trained weights, so post-hoc explanation methods are needed to understand why a model reached a particular decision for a particular entity. This project addresses both halves of that problem — training a competitive relational GNN and explaining its predictions — on a real, messy knowledge graph rather than a synthetic benchmark.

We use DBpedia as our data source and define a four-way node classification task: given a person entity and its local neighbourhood in DBpedia, predict which of four occupation categories (Athlete, Politician, Artist, Scientist) the person belongs to. Relational Graph Convolutional Networks (R-GCNs) [1] are well suited to this setting because they learn a separate transformation per relation (edge) type, using basis decomposition to keep the parameter count tractable even when the graph contains dozens of distinct DBpedia predicates.

A substantial part of this project turned out to be about a problem we did not anticipate: our first working pipeline achieved 100% accuracy on both the training and test sets — not a result to be proud of on a genuine four-way classification task, but a symptom of the model being handed the answer. Section 3.2 describes how we diagnosed and fixed this leakage in two rounds, the second of which required a

controlled ablation study rather than a single patch. We treat this investigation as one of the project's main technical contributions rather than a bug fixed quietly off-page.

The remainder of the report proceeds as follows. Section 2 describes the dataset and how it was constructed. Section 3 covers the R-GCN architecture, the leakage investigation and fix, and the model's final performance. Section 4 is the core of the report: how we explain the model's predictions with three complementary methods, and how those explanations were evaluated. Section 5 concludes, and Section 6 lists individual contributions.

2 Data Analysis

2.1 Extraction

We queried the public DBpedia SPARQL endpoint (<https://dbpedia.org/sparql>) for up to 400 person entities per occupation class, using the pattern “?person a <dbo:{Class}>” for each of the four target classes (dbo:Athlete, dbo:Politician, dbo:Artist, dbo:Scientist). All four classes returned the full quota, giving 1,600 labelled persons with a uniform 25% class prior. For each person we then fetched triples for a fixed set of relational predicates in batches of 50 entities per SPARQL query, since a single query spanning all 1,600 entities is impractical on the public endpoint. Results (persons and triples) were cached locally so that later pipeline stages and repeated experiments did not need to re-query DBpedia every time.

2.2 Predicates and graph construction

The final graph uses 19 realized relational predicates (out of 21 candidates queried; the remainder returned no triples for this sample): birthPlace, nationality, occupation, field, knownFor, genre, sport, party, team, influenced, associatedBand, almaMater, award, spouse, child, parent, deathPlace, religion, restingPlace, residence, and notableWork. Each predicate is added to the graph together with its inverse direction, doubling the number of relation types the R-GCN's basis decomposition has to learn ($19 \times 2 = 38$ relation types in the final graph). Notably, `rdf:type` is deliberately excluded from this list; Section 3.2 explains why in detail. Node features are a one-hot encoding of each node's out-degree, sized to the actual maximum degree observed in the graph rather than assumed from the node count (see the degree-cap fix in Section 3.2).

Metric	Value
Total nodes	4,475
Total directed edges (incl. inverse)	8,788
Relation types	38 (19 predicates $\times$ 2 directions)
Node feature dimension	25 (one-hot degree)
Labelled persons	1,600 (400 per class)
Training / test nodes	1,120 / 480 (70% / 30%)

Table 1: Final dataset statistics, after the leakage fix and predicate expansion described in Section 3.

2.3 Class distribution

The 1,600 persons are evenly split across the four classes by construction (400 each). The train/test split preserves this balance closely per class, as shown in Table 2.

Class	Train	Test	Total
Athlete	296	104	400
Politician	280	120	400
Artist	274	126	400
Scientist	270	130	400

Table 2: Class distribution across the train/test split.

3 Model Training and Evaluation

3.1 Architecture

We use a Relational GCN (FastRGCNConv in PyTorch Geometric) with basis decomposition (30 bases) to keep the parameter count manageable across 38 relation types. The final architecture, selected via the hyperparameter sweep described in Section 3.3, has three convolutional layers with 128 hidden channels, ReLU activations, and dropout ($p=0.5$) between layers, trained with Adam (learning rate 0.01, weight decay $5e-4$) for 400 epochs.

3.2 The accuracy problem: diagnosing label leakage

Our first working pipeline reached 100.00% train and 100.00% test accuracy on the four-way classification task. This is not a plausible outcome: chance accuracy is 25%, and comparable models on similar knowledge-graph classification tasks discussed in the course (e.g. CELOE and EvoLearner on the AIFB dataset) achieve accuracies in the 65–73% range, with even the strongest subgraph-based explainers reaching at most the high 80s. A perfect score on a non-trivial four-way task is a red flag for label leakage, not a sign of a good model.

The root cause was straightforward once found: DBpedia encodes class membership itself as an `rdf:type` triple (e.g. `<LeBron_James> rdf:type dbo:Athlete`), which is exactly how we assigned ground-truth labels when fetching persons. Our relational predicate list also included `rdf:type` without excluding these label-defining triples, so the very triple that determines the label was being re-fetched and added to the graph as an edge — handing the model the answer directly as an input feature rather than requiring it to learn from context.

Fix 1 excluded exactly the four label-defining triples — (person, `rdf:type`, <one of the four occupation class URIs>) — while keeping all other `rdf:type` triples, on the assumption that a person’s other types (e.g. `dbo:Person`) are legitimate structural context rather than a duplicate of the label. This reduced accuracy only slightly, to 99.91% train / 99.79% test — still implausibly high.

To find the remaining cause rather than guess at it, we ran a controlled ablation study: five conditions, built from the same cached triples, using the same random seed for both the train/test split and model initialization, and trained with the same baseline architecture (2 layers, 32 hidden channels, no dropout,

100 epochs), so that the only difference between conditions is which predicates are present. Table 3 reports the results.

Condition	Predicates removed	Train Acc.	Test Acc.
A: Baseline (all predicates, post Fix 1)	—	99.91%	99.79%
B: minus party/sport/team/genre/associatedBand/field	1,530 triples	99.91%	100.00%
C: minus ALL rdf:type triples	28,876 triples	74.82%	62.92%
D: minus occupation predicate	163 triples	99.82%	100.00%
E: minus rdf:type AND occupation	29,039 triples	73.48%	65.21%

Table 3: Five-condition ablation study isolating the source of residual accuracy after Fix 1.

Conditions B and D show no meaningful change from the baseline (B’s test accuracy even increased slightly), ruling out the hypothesis that “class-exclusive” predicates such as party or sport were driving the result. Conditions C and E, in contrast, both show a drop of roughly 35–37 percentage points in test accuracy, isolating rdf:type as the true remaining source of leakage — even though we had already removed the four exact label-defining triples. The explanation is that DBpedia additionally types every person with a redundant hierarchy of near-synonymous classes drawn from YAGO, Wikidata, and WordNet mappings (for example, a politician may also carry a fine-grained YAGO type such as “Politician110451863”) — different URIs from the four occupation classes we had excluded, but so strongly correlated with the label that they function as a near-duplicate of it.

Fix 2, the definitive fix, removes rdf:type from the relation set entirely, rather than attempting to hand-curate which specific type values are “safe.” During this investigation we also found and fixed a second, unrelated bug: the one-hot degree feature encoding assumed the maximum node degree could not exceed $N-1$ (valid only for a simple graph), but because every triple contributes both a forward and an inverse edge, a single well-connected “hub” object node can have out-degree exceeding $N-1$, which caused an index-out-of-bounds crash once the ablated (smaller) graphs were tested. The fix computes the true maximum degree present in the graph rather than assuming it equals $N-1$.

3.3 Recovering accuracy legitimately

After Fix 2, the honest, non-leaking baseline reaches 74.82% train / 62.92% test accuracy — roughly 2.5× better than the 25% random baseline on a genuinely hard four-way task. We treated this as a defensible result on its own, but pursued two further, legitimate (non-shortcut) improvements rather than reintroducing any label-adjacent feature. First, we added ten predicates that are informative but not tautologically tied to occupation in DBpedia’s schema (almaMater, award, spouse, child, parent, deathPlace, religion, restingPlace, residence, notableWork); on the same 2-layer/32-hidden architecture, this alone raised test accuracy to 73.12% (train 85.71%). Second, we ran a controlled hyperparameter sweep over model capacity and regularization — hidden width (32/64/128), depth (2 vs. 3 layers), dropout (0–0.5), and training length (100–300 epochs) — on this expanded, still-honest dataset.

Hidden	Layers	Dropout	Epochs	Train Acc.	Test Acc.
32	2	0.0	100	85.71%	73.12%

Hidden	Layers	Dropout	Epochs	Train Acc.	Test Acc.
64	2	0.0	100	85.89%	72.50%
64	2	0.3	200	85.71%	73.54%
128	2	0.3	200	85.80%	72.92%
64	3	0.3	200	86.16%	73.96%
128	3	0.5	300	86.16%	74.79%
64	2	0.5	300	85.80%	72.71%

Table 4: Hyperparameter sweep on the honest, expanded dataset. The bolded configuration (128 hidden, 3 layers, dropout 0.5, 300 epochs) was selected as final.

The best configuration — 128 hidden channels, 3 layers, dropout 0.5 — reached 74.79% test accuracy (86.16% train) at 300 epochs in the sweep, without the train/test gap widening substantially, which suggests genuine generalization rather than overfitting on the still-small graph. We adopt this configuration as our final model, extending training to 400 epochs, which reached 86.25% train / 73.75% test accuracy — consistent with the sweep result within normal run-to-run variation.

3.4 Final model performance

Table 5 summarizes the full trajectory of this investigation from the original leaking pipeline to the final, honest, tuned model. We consider this progression — not the final accuracy number in isolation — to be the project’s central technical finding: a high accuracy score is not, by itself, evidence of a good model, and confirming what a model actually relies on is inseparable from the broader goal of explainability.

Stage	Train Acc.	Test Acc.
Original pipeline (exact-label rdf:type leak present)	100.00%	100.00%
Fix 1: exact class-URI triples excluded	99.91%	99.79%
Fix 2: rdf:type removed entirely (honest baseline)	74.82%	62.92%
+ 10 additional legitimate predicates	85.71%	73.12%
+ tuned architecture (final model, 400 epochs)	86.25%	73.75%

Table 5: Accuracy across every stage of the leakage investigation and legitimate recovery.

4 Model Explanation

This section explains the final model’s predictions with three complementary methods — two established local explainers and one from-scratch implementation — and evaluates all three quantitatively.

4.1 Explanation methods

GNNExplainer [2] learns a soft mask over edges by optimizing mutual information between the masked subgraph’s prediction and the model’s original prediction for a given node. PGExplainer [3] instead trains a single shared MLP across many nodes to predict edge importance, amortizing the cost of explanation

across the dataset rather than re-optimizing separately for each node. Rather than use PyTorch Geometric’s built-in SubGraphX, we implemented a Monte Carlo Shapley-value approximation from scratch: for each edge incident to a target node, we sample random coalitions of neighbouring edges and estimate the edge’s Shapley value as the average change in the model’s confidence for the predicted class with versus without that edge present, across many random coalitions. All three methods convert their raw node/edge indices back into human-readable subject–predicate–object triples before display, so that explanations reference real entities and relations (e.g. “Brian Bennett → birthPlace → Harare”) rather than opaque node identifiers.

4.2 Qualitative examples

We explain one test node per class. Table 6 shows the single most important edge identified by each method for three representative entities; all three methods were correct on these predictions.

Entity (true / predicted class)	GNNExplainer top edge	SubGraphX top edge
Brian Bennett, cricketer (Athlete / Athlete)	birthPlace → Harare	birthPlace → Harare
Greg Stemen (Politician / Politician)	party → North Dakota Republican Party	party → North Dakota Republican Party (inverse)
Dijon Duenas (Artist / Artist)	genre → Alternative R&B	genre → Alternative R&B (inverse)

Table 6: Top explanatory edge per method for three example entities. PGExplainer agrees on which edges are selected but assigns them near-zero raw importance scores (see Section 4.4).

These are exactly the kind of intuitive, human-checkable justifications the task calls for: a politician explained by their party affiliation, a musician explained by their genre, an athlete explained by their birthplace. This is only possible because the leaking `rdf:type` feature was removed (Section 3.2) — the earlier, leaking version of the model would have trivially pointed to a single type edge on every node, which would have made this entire evaluation uninteresting.

4.3 Quantitative evaluation

We evaluate all three explainers with four metrics: Fidelity+ (the drop in the model’s confidence when the top-k explanation edges are removed — higher means those edges were truly important), Fidelity− (the model’s confidence when only the top-k explanation edges are kept — higher means the explanation alone is sufficient), Sparsity (the fraction of all graph edges retained in the explanation — lower is more concise), and Stability (mean pairwise Jaccard similarity of the top-k edge set across three independent GNNExplainer runs with different seeds — higher means the explanation is reproducible).

Explainer	Fidelity+	Fidelity−	Sparsity	Stability
GNNExplainer	1.4060	−0.1915	0.0005	1.0
PGExplainer	0.1581	−1.4394	0.0005	N/A
SubGraphX	1.4060	−0.1915	0.0004	N/A

Table 7: Explanation evaluation, averaged over three explained test nodes.

Two methodological notes on interpreting Table 7. First, Fidelity+ and Fidelity- are computed directly on the model's log-softmax output (log-probabilities), not on plain probabilities, so values are not restricted to the $[0, 1]$ range one might expect — a Fidelity+ above 1 reflects a large log-probability gap, not an error. Second, Sparsity is extremely low (well under 0.1% of all graph edges) because only the top-6 edges are retained as an explanation out of thousands of edges in the full graph; this is a desirable property (concise, focused explanations) rather than a defect. Stability, evaluated only for GNNExplainer, is perfect (1.0) on the tested node, indicating a highly reproducible explanation; we did not extend this evaluation to PGExplainer and SubGraphX given time constraints, which we flag as a direction for future work rather than gloss over.

4.4 Discussion: agreement and disagreement across methods

GNNExplainer and SubGraphX agree closely, both on which edges matter for a given entity and on their relative ranking — unsurprising, since both are optimization-based methods that directly measure the effect of removing an edge on the model's output. PGExplainer identifies the same edges as important in its top-6 ranking, but its raw importance scores are close to zero in absolute terms. We read this as PGExplainer's shared MLP being undertrained given the small number of nodes and epochs used for its explainer-training loop, rather than as evidence that those edges are unimportant — the ranking still agrees, only the score magnitudes are uninformative. We report this limitation honestly rather than hide it, in the same spirit as flagging the Fidelity metric's log-probability scale above.

5 Conclusion

We built an R-GCN pipeline for occupation classification on a DBpedia subgraph and explained its predictions with three complementary methods. The most consequential part of the project turned out to be diagnostic rather than architectural: our first working model reached a suspicious 100% accuracy, which we traced first to an exact label-leaking triple and then, after an initial fix left accuracy still implausibly high at 99.8%, to DBpedia's broader redundant type hierarchy via a controlled, five-condition ablation study. Removing the leaking feature entirely dropped accuracy to an honest 62.9%, which we then legitimately raised to 73.8% by adding genuinely informative predicates and tuning the model's capacity and regularization — without reintroducing any shortcut. The resulting model's predictions are explained consistently across GNNExplainer, PGExplainer, and a custom Shapley-based SubGraphX implementation, producing concise, human-readable justifications a domain expert could sanity-check by eye. We see this leakage investigation as the project's main contribution: a concrete demonstration that a high accuracy number is not, by itself, evidence of a good model, and that verifying what a model is actually keying on is inseparable from the broader goal of explainability.

6 Contributions of Team Members

[Team Member 1 Name] led — [describe: e.g. data extraction, SPARQL queries, dataset statistics].

[Team Member 2 Name] led — [describe: e.g. R-GCN implementation, the leakage investigation and ablation study, hyperparameter tuning].

[Team Member 3 Name] led — [describe: e.g. the three explanation methods, their quantitative evaluation, and visualizations].

All team members jointly discussed results, cross-checked numbers, and contributed to writing and revising this report.

Declaration on the Use of AI Tools

In preparing this project we used Claude (Anthropic) as an AI assistant for code development and report writing. For code development, Claude assisted with debugging the data pipeline, diagnosing the label leakage described in Section 3.2 (including designing and implementing the controlled ablation study), fixing the degree-cap bug, implementing the hyperparameter sweep, and adding the additional predicates used to legitimately recover accuracy. For report writing, Claude assisted with structuring this report and drafting prose from the experiment results and console output produced during development. In both cases the AI tool acted as an assistant rather than an author: all decisions about which hypotheses to test, which fixes to accept, and how to interpret results were made by the authors, and all reported numbers were produced by the authors' own runs of the code. The authors take full responsibility for all technical content, numbers, and claims in this report.

References

1. Schlichtkrull, M., Kipf, T.N., Bloem, P., van den Berg, R., Titov, I., Welling, M.: Modeling relational data with graph convolutional networks. In: ESWC 2018, pp. 593–607. Springer (2018).
2. Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: GNNExplainer: Generating Explanations for Graph Neural Networks. In: NeurIPS (2019).
3. Luo, D., Cheng, W., Xu, D., Yu, W., Zong, B., Chen, H., Zhang, X.: Parameterized Explainer for Graph Neural Network (PGExplainer). In: NeurIPS (2020).
4. Yuan, H., Yu, H., Wang, J., Li, K., Ji, S.: On Explainability of Graph Neural Networks via Subgraph Explorations (SubGraphX). In: ICML (2021).
5. DBpedia Association: DBpedia SPARQL endpoint. <https://dbpedia.org/sparql>, accessed 2026.